



Module Title

Cybersecurity and Networking

Internship Assignment Submission

Cookies, Sessions & Browser Security Fundamentals

Student Name: Jiwosh Shrestha

Internship ID: JVL-CS-2026-8990

Role: Cybersecurity Intern

Duration: 3 months

Assignment Submission Date: Sturday, 26th September 2026

Day:19 (Nineteen)

Contents

1.THEORY	3
1.1 Answer all 20 questions in your own words.....	3
2.PRACTICAL	6

1.THEORY

1.1 Answer all 20 questions in your own words

1.Why does a web application need state-management mechanisms if HTTP is stateless

HTTP treats each request separately. Cookies and sessions help the website remember the user between different requests.

2. What is a cookie?

A cookie is a small piece of data stored by the browser for a website. It can store things like preferences or session information.

3. What is Set-Cookie?

Set-Cookie is a response header sent by the server to tell the browser to store a cookie.

4. What is the Cookie request header?

The cookie header is sent by the browser to the server and contains cookies that match the request.

5. Explain the basic cookie request/response flow

The browser requests a website – server sends set cookie – browser stores the cookie – later the browser sends the cookie back to the server.

6. What is a web session?

A web session is a way for a website to remember a user's state across multiple requests.

7. What is a session ID?

A session ID is a unique identifier that helps the server identify a user's active session.

8. Explain the basic login-session flow.

The user logs in – server verifies the details – server creates a session – browser receives a session cookie – later request use the cookie to identify the session.

9. Why should sessions expire?

Sessions should expire to reduce the risk of an old or unsend session remaining active forever.

10. What happens conceptually during logout?

During logout, the server normally ends or invalidates the user's session. The related cookie may also be removed or expired.

11. What is the Secure cookie attribute?

Secure tells the browser to send the cookie only over secure HTTPS connections.

12. What is HttpOnly?

HttpOnly prevents normal Java Script, such as document. Cookie, from reading the cookie.

13. What is SameSite?

SameSite controls when a cookie can be sent with cross-site requests.

14. Explain Strict, Lax and None at a beginner level.

They control how long a cookie remains stored. Expires uses a date/time, while Max-Age uses a number of seconds.

15. What do Expires and Max-Age control?

They control how long a cookie remains stored. Expires uses a date/time, while Max-Age uses a number of second.

16. What is Path used for?

Path controls which URL path the browser should send the cookie to. For example, Path=/account limits it to matching account path.

17. What is local Storage?

Local Storage is browser storage used to save data for a website. The data can remain after the browser is closed and is not automatically sent with HTTP requests.\

18. What is session Storage?

Session Storage is browser storage that usually lasts for the current tab or page session.

19. Why is session Storage different from a server-side login session?

Session storage stores data in the browser, while a server-side login session is maintained by the web server to track an authenticated user.

20. Why should real session cookies never be shared in assignments or screenshots?

Real session cookies can be sensitive and may allow access to an active session. Therefore, assignments should use training values or property redacted cookies instead.

2.PRACTICAL

Document this training cookie: Set-Cookie: training_session=JVL123; Secure; HttpOnly; SameSite=Lax; Max-Age=3600; Path=/ Record: Cookie Name · Cookie Value · Secure Status · HttpOnly Status · SameSite Value · Max-Age · Path. Then write 3-5 sentences explaining why these attributes matter.

https://example.com

GET example.com / HTTP/2 HTTP/1.1

200 OK text/html; charset=UTF-8

Dashboard Proxy Target Repeater

Forward Drop Intercept is on

Target: https://example.com

GET / HTTP/1.1

Host: example.com

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Upgrade-Insecure-Requests: 1

Sec-Fetch-Dest: document

Sec-Fetch-Mode: navigate

Sec-Fetch-Site: none

Sec-Fetch-User: ?1

Connection: close

Intercept HTTP history WebSockets history Proxy settings

#	Host	Method	URL	Status	Length	MIME type	Title
1	example.com	GET	/	200	1,256	HTML	Example Domain
2	example.com	GET	/favicon.ico	404	285	HTML	404 Not Found

Dashboard Proxy Target Repeater

Send

Tab 1: https://example.com:443

Status: 200 OK (52 ms)

--- REQUEST (Repeater) ---

GET / HTTP/1.1

Host: example.com

User-Agent: Mozilla/5.0

Accept: text/html

Connection: close

--- RESPONSE (200 OK) ---

HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8

Content-Length: 1256

Server: ECS (dcb/7F3B)

<!doctype html>

<html><head><title>Example Domain</title>

https://example.com